ДИСЦИПЛИНА	Технологии разработки приложений на базе фреймворков
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическая работа
ПРЕПОДАВАТЕЛЬ	Ящун Татьяна Викторовна
СЕМЕСТР	5 семестр, 2026-2027

ПРАКТИЧЕСКАЯ РАБОТА № 5
DJANGO, HTTP И ОСНОВЫ ВЕБ-РАЗРАБОТКИ

ЦЕЛЬ РАБОТЫ

Изучить назначение фреймворка Django, структуру Django-проекта и Django-приложения, принципы обработки HTTP-запросов и формирования HTTP-ответов, маршрутизацию URL и основы создания веб-страниц.

На основе объектной модели индивидуального проекта, разработанной в ПР3, подготовить проект к переходу от консольного приложения к веб-приложению на Django.

СОДЕРЖАНИЕ

1. ЗАДАЧИ РАБОТЫ 2
2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ 3
  2.1. Подготовка к работе 3
  2.2. Исходное состояние индивидуального проекта 3
  2.3. Контрольная точка Git 4
3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 4
  3.1. Веб-приложение и протокол HTTP 4
  3.2. Фреймворк и Django 5
  3.3. Django-проект и Django-приложение 6
  3.4. Структура Django-проекта 6
  3.5. Маршрутизация URL и view-функции 7
  3.6. HTML, CSS и Bootstrap 8
4. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА 9
  4.1. Введение и создание Django-проекта 9
  4.2. HTTP и маршрутизация 10
  4.3. HTML, CSS и Bootstrap 10
5. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ 11
  5.1. Подготовка рабочего проекта 11
    5.1.1. Используется проект из ПР3 11
    5.1.2. Проверка состояния проекта 11

5.1.3. Создание контрольной точки Git 12
5.1.4. Анализ переноса сценариев в веб-интерфейс 12
5.1.5. Определение страниц проекта 12
5.1.6. Результат первой части практической работы 12
5.2. Требования к доработке индивидуального проекта 13
5.3. Что изменяется по сравнению с ПР3 13
5.4. Последовательная доработка проекта 14
5.4.1. Подготовка проекта к переходу на Django 14
5.4.2. Создание Django-проекта 15
5.4.3. Создание Django-приложений 16
5.4.4. Первые view-функции 18
5.4.5. Настройка маршрутизации 20
5.4.6. Страницы проекта 22
5.4.7. Обработка ошибки 404 24
5.4.8. Данные проекта ПР3 в веб-интерфейсе 26
5.5. Обновление README.md 33
5.6. Проверка и фиксация результата 39
5.6.1. Проверка Django-приложения 39
5.6.2. Проверка маршрутов и страниц 40
5.6.3. Проверка качества кода 41
5.6.4. Проверка изменений Git 42
6. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ 42
7. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ 43
7.1. Чек-лист выполнения материалов Яндекс Практикума 43
7.2. Чек-лист выполнения индивидуального проекта 44
7.3. Критерии оценки 44
7.4. Требования к отчету 45
8. КОНТРОЛЬНЫЕ ВОПРОСЫ 46
9. ИТОГ 47

1. ЗАДАЧИ РАБОТЫ

В рамках практической работы необходимо:

- выполнить материалы Яндекс Практикума по разделу 04 «Основы веб-разработки»;
- изучить назначение и основные возможности фреймворка Django;
- создать Django-проект и Django-приложения;
- изучить структуру Django-проекта;
- изучить назначение HTTP и основные элементы HTTP-запроса и HTTP-ответа;
- настроить маршрутизацию URL;
- реализовать обработчики запросов – view-функции;
- изучить основы HTML и CSS, необходимые для создания веб-интерфейса;
- подключить Bootstrap для оформления веб-страниц;
- создать основу веб-интерфейса индивидуального проекта;
- сохранить предметную область индивидуального проекта, разработанную в ПР3;
- подготовить структуру проекта к дальнейшему использованию Django ORM и шаблонов;

- проверить работоспособность приложения;
- обновить README.md;
- зафиксировать результат работы в Git с обязательной меткой текущей практической работы PR5.

2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ

2.1. Подготовка к работе

Перед началом работы:

- войти в личный аккаунт Яндекс, используемый для обучения;
- открыть курс Яндекс Практикума «Бэкенд-разработка»;
- перейти к разделу 04 «Основы веб-разработки»;
- проверить состояние индивидуального проекта после ПР3;
- убедиться, что проект запускается, тесты выполняются, а изменения ПР3 зафиксированы в Git;
- ознакомиться с целью, задачами и требованиями настоящей практической работы.

ПР5 является логическим продолжением ПР3: переход к веб-разработке выполняется в том же репозитории индивидуального проекта. Создавать новый репозиторий не требуется.

Примечание. ПР4 (контрольная работа) не изменяет индивидуальный проект. Поэтому исходным состоянием ПР5 является проект в состоянии после ПР3: классы Room, User и Booking, JSON-хранилище, тесты и консольная версия приложения.

2.2. Исходное состояние индивидуального проекта

Перед началом ПР5 индивидуальный проект должен содержать:

- предметную область, определенную в ПР1–ПР3;
- классы `Room`, `User` и `Booking` (пакет `models/`);
- функции работы с сущностями и пользовательскими сценариями;
- хранение данных в JSON-файлах (`data/rooms.json`, `data/users.json`, `data/bookings.json`);
- автоматизированные тесты (`tests/`, pytest);
- проверку качества кода (flake8);
- `README.md`;
- Git-репозиторий.

Для проекта `Roomly` исходным состоянием является консольное приложение с объектами `Room`, `User` и `Booking`. Сущности связаны между собой: бронирование (`Booking`) ссылается на помещение (`Room`) и пользователя (`User`).

В ПР5 переход к Django ORM еще не выполняется: основное внимание уделяется структуре веб-приложения, HTTP, URL, view-функциям и формированию HTML-

страниц. Классы ПР3 продолжают использоваться как источник данных для веб-интерфейса.

Проект развивается от работы к работе (рис. 1): ПР5 добавляет к объектной модели веб-слой.

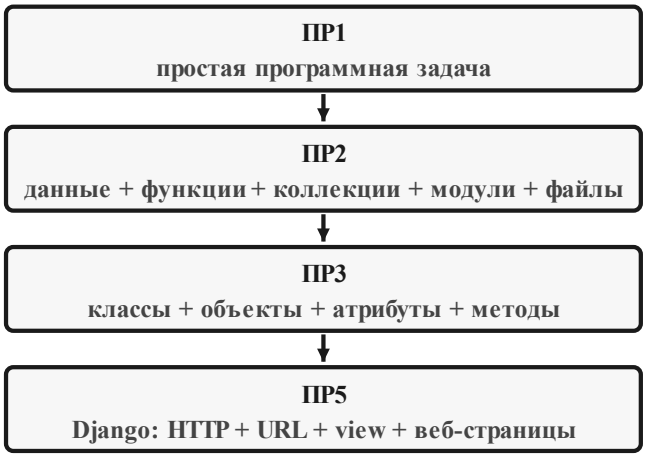


Рисунок 1 – Принцип развития индивидуального проекта

2.3. Контрольная точка Git

Перед началом переработки проекта необходимо проверить состояние репозитория:

```
git status
```

При необходимости сохранить текущее состояние отдельным коммитом:

```
git add .
git commit -m "PR5: Подготовка проекта к переходу на Django"
```

Каждый последующий коммит, относящийся к ПР5, должен начинаться с метки `PR5:`. Например:

```
PR5: create Django project
PR5: add room views
PR5: configure URL routing
```

3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

3.1. Веб-приложение и протокол HTTP

Веб-приложение – программная система, доступ к которой осуществляется через веб-браузер или другой HTTP-клиент. В отличие от консольного приложения, пользователь взаимодействует с веб-приложением посредством HTTP-запросов: вместо ввода данных через `input()` браузер отправляет запрос по определенному адресу, а программа возвращает ответ.

HTTP – протокол передачи гипертекста, стандарт обмена данными между клиентом и сервером. Обмен строится по схеме «запрос – ответ» (рис. 2): клиент отправляет HTTP-запрос, сервер обрабатывает его и формирует HTTP-ответ.

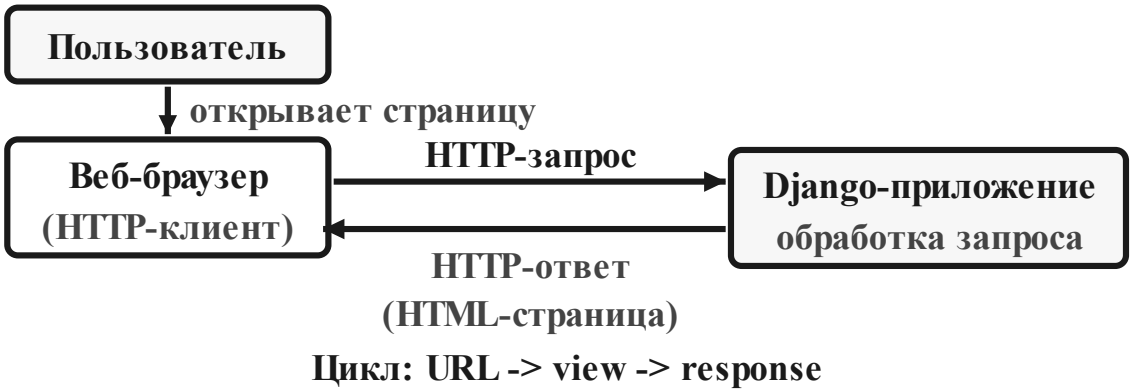


Рисунок 2 – Цикл взаимодействия с веб-приложением

HTTP-запрос состоит из трех основных частей:

- стартовая строка – метод, адрес и версия протокола, например `GET /rooms/ HTTP/1.1`;
- заголовки – дополнительные сведения о запросе (`Host`, `User-Agent`, `Accept` и другие);
- тело – передаваемые данные (у GET-запросов обычно отсутствует).

HTTP-ответ включает:

- стартовую строку – версию протокола и код статуса, например `HTTP/1.1 200 OK`;
- заголовки (`Content-Type`, `Content-Length` и другие);
- тело – передаваемые данные, например HTML-страницу.

Код статуса сообщает результат обработки запроса: `200` – успешная обработка, `404` – страница не найдена, `500` – ошибка сервера. Для получения страниц используется метод GET, для отправки данных на сервер – метод POST.

В рамках ПР5 необходимо реализовать базовый цикл URL → view → response: обращение по определенному адресу должно приводить к формированию и возврату HTML-страницы.

3.2. Фреймворк и Django

Фреймворк предоставляет готовую структуру и набор инструментов для разработки приложений. В отличие от библиотеки, которую программист вызывает из собственного кода, фреймворк сам управляет ходом выполнения программы, а разработчик наполняет его своими модулями и обработчиками.

Django – веб-фреймворк для Python, предназначенный для создания веб-приложений. Django предоставляет средства для:

- обработки HTTP-запросов;
- маршрутизации URL;
- формирования HTTP-ответов;
- работы с HTML-шаблонами;
- работы с базами данных (Django ORM);
- обработки форм;
- аутентификации пользователей;
- администрирования приложения;
- тестирования;
- разработки API.

В методических указаниях используется **Django 5.2** – актуальная версия с долгосрочной поддержкой (LTS). Все команды, структура проекта и настройки соответствуют возможностям линейки 5.2 и будут работать и в последующих версиях этой линии. В ПР5 изучаются базовые механизмы Django, необходимые для создания первого веб-интерфейса индивидуального проекта.

3.3. Django-проект и Django-приложение

Важно различать два понятия:

- **Django-проект** – общая конфигурация системы: настройки, маршрутизация верхнего уровня, точки запуска;
- **Django-приложение** – функциональная часть системы, реализующая отдельную область.

Django-проект содержит настройки и общую конфигурацию веб-приложения. Django-приложение предназначено для реализации отдельной функциональной области. Например, для системы бронирования помещений в дальнейшем могут использоваться приложения `homepage`, `rooms` и `bookings`.

3.4. Структура Django-проекта

После создания Django-проекта формируется структура, включающая файл управления проектом и пакет с настройками. Типовая структура:

```
project/
│
├── manage.py
│
└── project/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

Элемент	Назначение
`manage.py`	запуск команд управления Django-проектом
`settings.py`	настройки проекта
`urls.py`	маршрутизация URL
`asgi.py`	точка взаимодействия с ASGI-сервером
`wsgi.py`	точка взаимодействия с WSGI-сервером
`__init__.py`	обозначение Python-пакета

При создании приложения появляется отдельный пакет приложения, например `rooms`:

```
rooms/
├── __init__.py
├── admin.py
├── apps.py
├── migrations/
├── models.py
├── tests.py
└── views.py
```

Назначение основных файлов приложения:

Файл	Назначение
`admin.py`	настройка панели администрирования
`apps.py`	конфигурация приложения
`migrations/`	миграции базы данных
`models.py`	модели данных Django
`tests.py`	тесты приложения
`views.py`	view-функции – обработчики запросов
`urls.py`	маршруты приложения; файл добавляется вручную

Примечание. Не следует путать файл `models.py` Django-приложения с пакетом `models/` проекта ПР3: пакет `models/` содержит классы предметной области `Room`, `User` и `Booking`, а `models.py` приложения пока остается пустым – Django-модели будут созданы на следующих практических работах.

3.5. Маршрутизация URL и view-функции

Маршрутизация – процесс определения обработчика по адресу запроса. Когда пользователь обращается к URL Django-приложения, запрос проходит по цепочке (рис. 3): файл `urls.py` проекта, маршруты приложения, *view*-функция, HTTP-ответ.

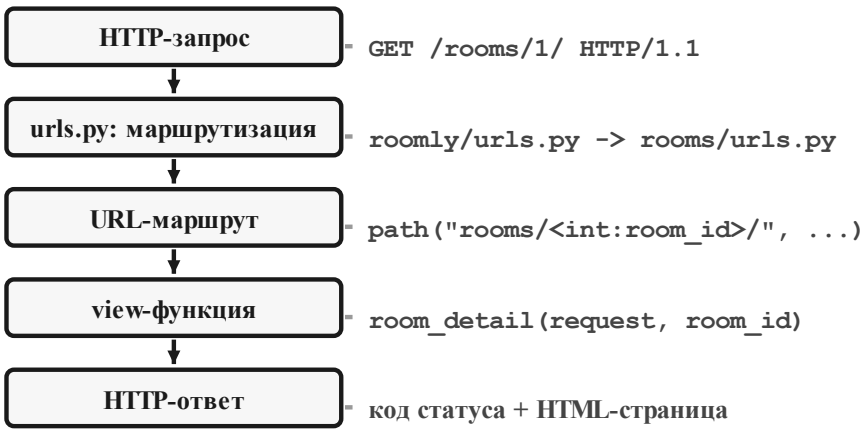


Рисунок 3 – Обработка запроса в Django

Маршрут описывается функцией `path()`: первый аргумент – шаблон адреса, второй – *view*-функция, третий (необязательный) – имя маршрута. Для динамических адресов используются конвертеры путей, например `<int:room_id>`: сегмент адреса преобразуется в целое число и передается в *view*-функцию параметром.

Маршруты распределяются по уровням (рис. 4): файл `urls.py` проекта подключает маршруты приложений функцией `include()`, а каждое приложение содержит собственный `urls.py` со своими маршрутами. Такой подход соответствует материалу «Распределение адресов по приложениям» и упрощает развитие проекта.

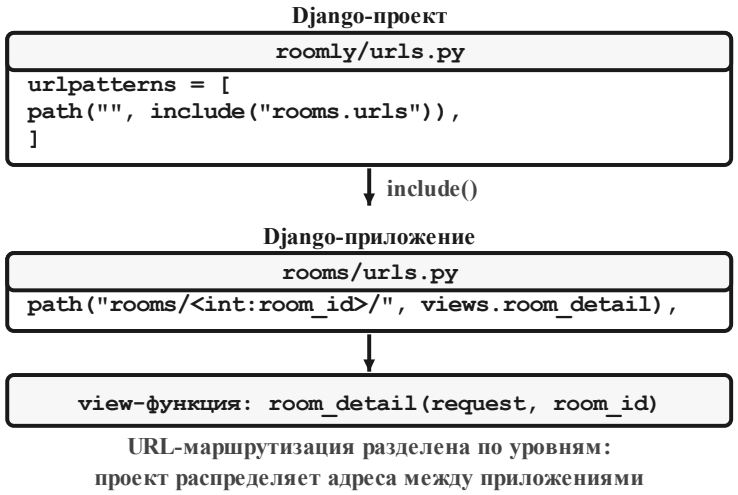


Рисунок 4 – Распределение URL между проектом и приложением

view-функция принимает объект запроса `request` и возвращает объект ответа `HttpResponse`. Формирование ответа – результат работы *view*: данные предметной области превращаются в HTML-страницу.

3.6. HTML, CSS и Bootstrap

HTML – язык разметки гипертекста, описывающий структуру веб-страницы. HTML-документ состоит из двух основных частей: `head` (служебная информация: кодировка, заголовок вкладки, подключение стилей) и `body` (содержимое страницы). Основные элементы: заголовки `h1`–`h6`, абзацы `p`, списки `ul`/`ol`/`li`, ссылки `a`, таблицы `table`.

CSS – язык описания внешнего вида страницы. Селекторы определяют, к каким элементам применяются стили; свойства задают оформление: цвет, размер, отступы. Стили наследуются – свойства родительских элементов переходят к вложенным элементам.

Bootstrap – CSS-фреймворк для быстрой верстки интерфейсов. Bootstrap содержит готовые компоненты (навигационная панель, кнопки, таблицы, карточки) и сетку для адаптивной верстки. В ПР5 используется актуальная версия **Bootstrap 5.3**, подключаемая через CDN без установки дополнительных пакетов.

4. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА

В рамках ПР5 выполняются материалы Раздела 04 «Основы веб-разработки» курса Яндекс Практикума «Бэкенд-разработка». Перечень материалов приведен в таблице: темы отмечаются по мере выполнения. Полный чек-лист для контроля – в подразделе 7.1.

№	Тема	Урок (материал)	Выполнить
1	Создание проекта	«Введение в спринт»	✓
2	Создание проекта	«Фреймворк Django»	✓
3	Создание проекта	«Создание Django-проекта»	✓
4	Создание проекта	«Приложения в Django-проекте»	✓
5	Пути и view-функции	«Протокол HTTP: стандарт для запросов и ответов»	✓
6	Пути и view-функции	«Планирование адресов и конвертеры путей»	✓
7	Пути и view-функции	«Распределение адресов по приложениям»	✓
8	Пути и view-функции	«View-функции: от запроса до ответа»	✓
9	Вёрстка для бэкендера	«Веб-страницы: язык HTML»	✓
10	Вёрстка для бэкендера	«Структура HTML-документа»	✓
11	Вёрстка для бэкендера	«CSS: стили, селекторы и синтаксис»	✓
12	Вёрстка для бэкендера	«Иерархия HTML-документа и наследование стилей»	✓
13	Вёрстка для бэкендера	«Фреймворк Bootstrap: быстрая HTML-верстка»	✓

4.1. Введение и создание Django-проекта

Выполнить материалы:

- «Создание проекта/Введение в спринт»;
- «Создание проекта/Фреймворк Django»;
- «Создание проекта/Создание Django-проекта»;
- «Создание проекта/Приложения в Django-проекте».

В результате необходимо:

- установить Django и проверить версию;
- создать Django-проект;
- запустить проект и проверить работу development-сервера;
- создать Django-приложения;
- зарегистрировать приложение в настройках проекта (INSTALLED_APPS);
- ознакомиться со структурой проекта и приложения;
- определить назначение основных файлов Django-проекта и приложения.

Установка выполняется в виртуальное окружение, созданное в ПР1. В `requirements.txt` проекта фиксируется версия Django:

```
pip install "Django==5.2.*"
```

Примечание. В `requirements.txt` добавляется строка `Django==5.2.*`. Указывайте версию явно: команда `pip install Django` без ограничения может установить другую мажорную версию фреймворка.

Проверка запуска проекта выполняется командой:

```
python manage.py runserver
```

После запуска необходимо открыть в браузере адрес `http://127.0.0.1:8000/` и убедиться, что *development*-сервер Django корректно обрабатывает запрос (отображается стартовая страница Django).

4.2. HTTP и маршрутизация

Выполнить материалы:

- «Пути и view-функции/Протокол HTTP: стандарт для запросов и ответов»;
- «Пути и view-функции/Планирование адресов и конвертеры путей»;
- «Пути и view-функции/Распределение адресов по приложениям»;
- «Пути и view-функции/View-функции: от запроса до ответа».

В рамках этих материалов необходимо изучить:

- назначение HTTP;
- структуру HTTP-запроса и HTTP-ответа;
- URL и маршрутизацию запросов;
- параметры URL и конвертеры путей;
- распределение URL между приложениями;
- view-функции и формирование HTTP-ответа.

Базовая схема обработки запроса в Django рассмотрена в подразделе 3.5 (рис. 3).

4.3. HTML, CSS и Bootstrap

Выполнить материалы:

- «Вёрстка для бэкендера/Веб-страницы: язык HTML»;
- «Вёрстка для бэкендера/Структура HTML-документа»;
- «Вёрстка для бэкендера/CSS: стили, селекторы и синтаксис»;
- «Вёрстка для бэкендера/Иерархия HTML-документа и наследование стилей»;
- «Вёрстка для бэкендера/Фреймворк Bootstrap: быстрая HTML-верстка».

В результате необходимо уметь:

- создавать корректную структуру HTML-документа;
- использовать заголовки, абзацы, списки, ссылки и таблицы;
- описывать стили CSS и объяснять их наследование;
- подключать Bootstrap и использовать готовые компоненты.

5. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ

5.1. Подготовка рабочего проекта

Первая часть практической работы выполняется до создания Django-проекта. На этом этапе проверяется работоспособность проекта из ПР3, создается контрольная точка в Git и определяются страницы будущего веб-интерфейса. Пункты 5.1.1–5.1.5 выполняются последовательно; их результат контролируется в пункте 5.1.6 перед переходом к подразделу 5.2.

5.1.1. Используется проект из ПР3

Перед началом работы необходимо открыть репозиторий индивидуального проекта, созданный в ПР1, доработанный в ПР2 и переведенный на объектную модель в ПР3. Создавать отдельный репозиторий для ПР5 не требуется. Структура проекта на начало ПР5 должна соответствовать результату ПР3. Для проекта `Roomly`:

```
room_booking/
│
├── README.md
├── requirements.txt
├── main.py
├── storage.py
├── utils.py
│
├── models/
│   ├── __init__.py
│   ├── rooms.py
│   ├── bookings.py
│   └── users.py
│
├── data/
│   ├── rooms.json
│   ├── bookings.json
│   └── users.json
│
└── tests/
    ├── test_rooms.py
    ├── test_bookings.py
    └── test_users.py
```

Для собственного проекта названия модулей и файлов будут отличаться, так как они должны соответствовать предметной области.

5.1.2. Проверка состояния проекта

Перед внесением изменений необходимо убедиться, что проект после ПР3 работоспособен. Проверить:

1. Запуск консольного приложения (`python main.py`)

2. Основные операции меню приложения

3. Загрузку данных из JSON-файлов

4. Сохранение данных

5. Выполнение автоматических тестов (`pytest -v`)

6. Проверку качества кода (`flake8 .`)

Если после ПР3 в проекте имеются ошибки, сначала необходимо определить их причину и зафиксировать исходное состояние.

5.1.3. Создание контрольной точки Git

Перед преобразованием проекта рекомендуется сохранить состояние в Git:

```
git status
git add .
git commit -m "PR5: Подготовка проекта к переходу на Django"
```

5.1.4. Анализ переноса сценариев в веб-интерфейс

Необходимо определить, какие операции консольного приложения должны получить веб-представление. Для `Roomly` к основным сценариям относятся:

- просмотр списка помещений;
- просмотр информации о конкретном помещении;
- проверка вместимости помещения;
- просмотр бронирований;
- проверка доступности помещения на дату.

На ПР5 реализуется прежде всего отображение информации. Создание, редактирование и удаление данных через веб-формы будут реализованы позднее, после изучения моделей и форм Django.

5.1.5. Определение страниц проекта

Для выбранных сценариев определяются страницы веб-интерфейса и их адреса. Минимальный набор страниц для `Roomly`:

Страница	URL	Назначение
Главная	`/`	описание приложения и навигация
Список помещений	`/rooms/`	отображение объектов предметной области
Страница помещения	`/rooms/<id>/`	информация об отдельном помещении
Список бронирований	`/bookings/`	бронирования проекта

Названия и состав страниц могут отличаться для других вариантов индивидуального проекта.

5.1.6. Результат первой части практической работы

После выполнения пунктов 5.1.1–5.1.5 должны быть получены:

- работоспособный проект в состоянии ПР3;
- контрольная точка Git с меткой `PR5:`;

- определенный перечень веб-страниц проекта и их адресов.

Только после этого следует переходить к доработке проекта.

5.2. Требования к доработке индивидуального проекта

К завершению ПР5 индивидуальный проект должен представлять собой работоспособное Django-приложение с базовым веб-интерфейсом. Необходимо:

- создать Django-проект;
- создать Django-приложение;
- подключить приложения к проекту;
- настроить маршрутизацию;
- реализовать *view*-функции;
- создать несколько веб-страниц;
- настроить переходы между страницами;
- использовать HTML;
- использовать CSS и Bootstrap;
- реализовать отображение данных предметной области;
- сохранить связь проекта с предметной областью ПР1–ПР3;
- обновить `README.md`;
- проверить запуск Django-приложения;
- зафиксировать изменения в Git.

5.3. Что изменяется по сравнению с ПР3

В ПР3 основным способом взаимодействия с программой было консольное меню. В ПР5 пользователь взаимодействует с приложением через веб-браузер (рис. 5).

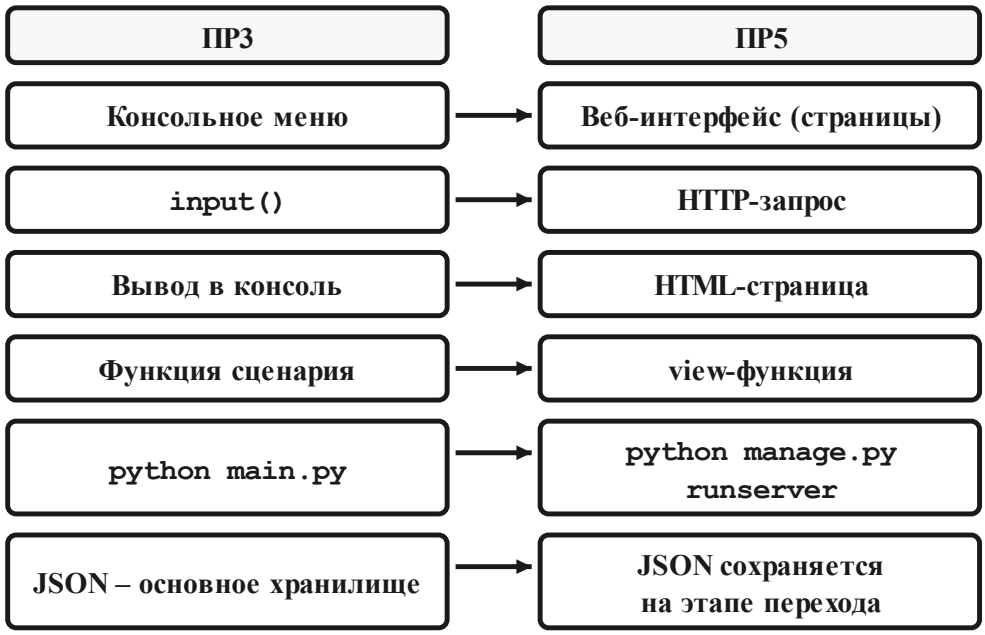


Рисунок 5 – Сравнение подходов ПР3 и ПР5

ПР3	ПР5
Консольное меню	Веб-интерфейс
`input()`	HTTP-запрос
Вывод в консоль	HTML-страница
Функция сценария	view-функция
Python-структуры данных	Подготовка данных для веб-страницы
Запуск `python main.py`	Запуск development-сервера Django
JSON как основное хранилище	JSON временно сохраняется на этапе перехода

На ПР5 не требуется удалять код ПР3. Необходимо сохранить работоспособность предыдущей версии проекта до тех пор, пока соответствующая функциональность не будет последовательно перенесена в Django: консольное приложение `main.py`, классы пакета `models/`, функции `storage.py` и тесты продолжают работать.

5.4. Последовательная доработка проекта

Индивидуальный проект переходит от консольного приложения к веб-приложению (рис. 6). Исходная архитектура ПР3: меню, функции сценариев, классы предметной области, JSON-хранилище. Целевая структура данного этапа: браузер, HTTP-запрос, маршрутизация, view-функция, HTML-страница.

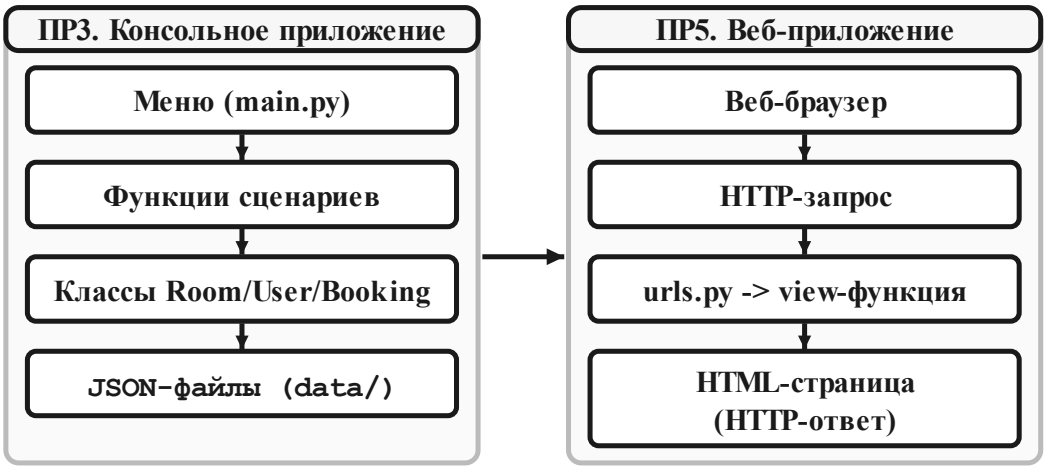


Рисунок 6 – Переход от консольного приложения к веб-приложению

Работа с базой данных и перенос классов в Django-модели выполняются на следующих практических работах. Пункты 5.4.1–5.4.8 выполняются последовательно.

5.4.1. Подготовка проекта к переходу на Django

На данном этапе выполняется доработка существующего проекта `Roomly` с переходом от консольного приложения к веб-приложению на Django.

Работа продолжается в существующем репозитории `room_booking`. Для разработки используется Python 3.11 и Django 5.2. Перед созданием Django-проекта необходимо подготовить виртуальное окружение и установить необходимые зависимости.

1. Проверьте наличие Python 3.11:

```
py --version
```

Команда должна вывести версию Python, начинающуюся с 3.x.

2. В корне репозитория `room_booking` создайте виртуальное окружение на основе:

```
py -m venv venv
```

В результате в корне репозитория будет создан каталог `venv`, содержащий виртуальное окружение проекта.

3. Активируйте виртуальное окружение:

```
venv\Scripts\activate
```

После успешной активации в начале строки терминала должно отображаться имя виртуального окружения:

```
(venv)
```

4. Проверьте версию Python в активированном виртуальном окружении:

```
python --version
```

Команда должна вывести версию Python 3.x.x.

5. Откройте файл `requirements.txt` и добавьте зависимость Django версии 5.2:

```
Django==5.2.*
```

6. Установите зависимости проекта:

```
pip install -r requirements.txt
```

7. Проверьте установку Django:

```
django-admin --version
```

и

```
python -m django --version
```

Команда должна вывести версию 5.2.x.

5.4.2. Создание Django-проекта

После успешной установки Django перейдите в корень репозитория `room_booking` и создайте Django-проект непосредственно в текущей директории:

```
django-admin startproject roomly .
```

Точка в конце команды указывает, что Django-проект создаётся не в новом каталоге, а в текущей директории. Поэтому файл `manage.py` размещается непосредственно в корне репозитория – рядом с существующими файлами `main.py`, `storage.py`, `utils.py` и каталогами `models/`, `data/`, `tests/`.

Обновленная структура репозитория после создания Django-проекта:

```
├── manage.py
├── roomly/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
├── models/
├── data/
├── tests/
├── main.py
├── storage.py
├── utils.py
└── requirements.txt
```

В данной структуре:

- `roomly/` – пакет конфигурации Django-проекта;
- `models/`, `data/` и `tests/` – существующие каталоги проекта `Roomly`, сформированные на предыдущих этапах разработки;
- `urls.py` приложений (`homepage/urls.py`, `rooms/urls.py`, `bookings/urls.py`) – файлы маршрутов Django-приложений; будут добавлены вручную на соответствующем этапе (пункт 5.4.5).

На этом этапе существующий код `Roomly` не удаляется: Django добавляется к уже существующему проекту, после чего функциональность приложения будет постепенно переноситься на новую архитектуру.

Примечание. Создавать отдельный каталог для Django-проекта не требуется: репозиторий индивидуального проекта уже существует с ПР1, поэтому `manage.py` размещается в его корне.

Каталог `roomly/` содержит конфигурацию Django-проекта:

- `settings.py` – основные настройки проекта;
- `urls.py` – корневую конфигурацию маршрутов;
- `asgi.py` и `wsgi.py` – точки входа для соответствующих серверов;
- `__init__.py` – признак Python-пакета.

В файле roomly/settings.py настройте параметры локализации:

```
LANGUAGE_CODE = "ru-RU"
TIME_ZONE = "Europe/Moscow"
USE_TZ = True
```

5.4.3. Создание Django-приложений

В проекте `Roomly` четыре предметные области: главная страница, помещения, бронирования и пользователи. Три из них получат собственные Django-приложения; для четвёртой приложение не создаётся.

Область	Приложение	Почему
Главная страница	`homepage`	Точка входа в систему, не относится к другим областям
Помещения	`rooms`	Отдельная сущность `Room` из ПР3
Бронирования	`bookings`	Отдельная сущность `Booking` из ПР3
Пользователи	–	Django предоставляет встроенную модель `User`

Для пользователей отдельное приложение не создаётся: Django уже предоставляет готовую модель `User` в приложении `django.contrib.auth`, подключённом в `INSTALLED_APPS` по умолчанию. Создавать своё приложение `user` значило бы конфликтовать с встроенным. Работа с пользователями (регистрация, вход, разграничение доступа) будет рассмотрена в следующих темах.

Главная страница логически не относится ни к помещениям, ни к бронированиям: она выполняет роль точки входа в систему. Если оставить её в приложении `rooms`, при росте проекта будет непонятно, куда размещать файлы, отвечающие за работу главной.

В терминале с активированным виртуальным окружением создать три приложения:

```
python manage.py startapp homepage
python manage.py startapp rooms
python manage.py startapp bookings
```

В результате в структуре проекта происходят изменения и появляются каталоги `homepage/`, `rooms/` и `bookings/`.

```
room_booking/
│
├── models/
│   ├── rooms.py
│   ├── bookings.py
│   └── users.py
├── data/
├── roomly/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── homepage/
│   ├── migrations/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
├── rooms/
│   ├── migrations/
│   ├── __init__.py
│   ├── admin.py
```

```
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
├── bookings/
│   ├── migrations/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
├── tests/
├── README.md
├── manage.py
├── main.py
├── storage.py
├── utils.py
└── requirements.txt
```

Зарегистрировать приложения в `roomly/settings.py`, добавив их имена в список `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "homepage",
    "rooms",
    "bookings",
]
```

Регистрация обязательна: без неё Django не обнаружит приложения и их компоненты. Файлы `homepage/urls.py`, `rooms/urls.py` и `bookings/urls.py` для маршрутов приложений создаются вручную (пункт 5.4.5).

Примечание. Модуль `models/users.py` из ПР3 остаётся в проекте как часть консольного приложения. Django-приложения `homepage`, `rooms` и `bookings` будут использовать `Room` и `Booking` из `models/rooms.py` и `models/bookings.py`; модуль `users.py` пока не задействован – он понадобится, когда в курсе появится тема аутентификации.

5.4.4. Первые view-функции

View-функция – обычная функция Python, которая принимает объект запроса `HttpRequest` и возвращает объект ответа `HttpResponse`. Первым аргументом view-функция всегда принимает `request`; если в шаблоне адреса использована переменная, она передаётся вторым аргументом.

HttpResponse принимает первым аргументом строку – текстовое содержимое, которое отобразится в браузере. Параметр status задаёт код ответа сервера (200 – успех, 404 – не найдено).

View-функции хранятся в файле views.py каждого приложения – по той же логике, что и urls.py.

Первоначальная проверка маршрутизации выполняется view-функциями, возвращающими короткие HttpResponse. Опишите в файлах views.py трёх приложений следующие функции. В файле homepage/views.py:

```
from django.http import HttpResponse


def index(request):
    return HttpResponse("Roomly – система бронирования помещений")
```

В файле rooms/views.py:

```
from django.http import HttpResponse


def rooms(request):
    return HttpResponse("Список помещений")


def room_detail(request, room_id):
    return HttpResponse(f"Помещение {room_id}")
```

В файле bookings/views.py:

```
from django.http import HttpResponse


def bookings(request):
    return HttpResponse("Список бронирований")


def booking_detail(request, booking_id):
    return HttpResponse(f"Бронирование {booking_id}")
```

Данные функции являются первоначальной проверкой прохождения запроса по цепочке: браузер → URL → urls.py → view → response. Переносить всю функциональность консольного приложения в Django на этом этапе не требуется.

Обратите внимание: функции room_detail() и booking_detail() принимают второй аргумент – room_id и booking_id соответственно. Этот аргумент будет передан из URL в пункте 5.4.5, где в шаблонах адресов появятся переменные.

Даже если view-функция не использует этот аргумент, она обязана его принять – иначе Django вернёт ошибку.

5.4.5. Настройка маршрутизации

Маршруты распределяются по двум уровням. В корневом файле roomly/urls.py подключаются маршруты приложений; в файлах urls.py самих приложений описываются конкретные адреса.

В файле roomly/urls.py подключите маршруты трёх приложений:

```
from django.contrib import admin
from django.urls import include, path


urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("homepage.urls")),
    path("rooms/", include("rooms.urls")),
    path("bookings/", include("bookings.urls")),
]
```

Функция include() «режет» URL на две части: часть адреса указывается в корневом urls.py, оставшаяся часть – в urls.py приложения. Например, при запросе к /rooms/1/ в корневом файле сработает path("rooms/", include("rooms.urls")), а в rooms/urls.py Django будет искать шаблон, соответствующий адресу 1/ – без префикса rooms/.

Создайте файлы urls.py в приложениях вручную (Django не создаёт их автоматически). В файле homepage/urls.py:

```
from django.urls import path

from . import views


urlpatterns = [
    path("", views.index, name="index"),
]
```

В файле rooms/urls.py:

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.rooms, name="rooms"),
    path("<int:room_id>/", views.room_detail, name="room_detail"),
]
```

В файле bookings/urls.py:

```
from django.urls import path

from . import views


urlpatterns = [
    path("", views.bookings, name="bookings"),
    path("<int:booking_id>/", views.booking_detail, name="booking_detail"),
]
```

Имена маршрутов (name=) потребуются в следующих работах при переходе на шаблоны: ссылки будут формироваться по именам маршрутов, а не жёстко заданными адресами.

Соответствие маршрутов и view-функций:

URL	View	Результат
/	homepage.views.index	главная страница
/rooms/	rooms.views.rooms	список помещений
/rooms/1/	rooms.views.room_detail	информация о помещении
/bookings/	bookings.views.bookings	список бронирований
/bookings/1/	bookings.views.booking_detail	информация о бронировании

Django читает список urlpatterns сверху вниз и вызывает обработчик из первого же path(), в котором шаблон адреса совпадёт с запросом. Если запрошенный URL не обнаружился в списке, пользователю отправляется страница с сообщением об ошибке 404 Page Not Found. Как заменить её собственной страницей проекта – в разделе 5.4.7.

При планировании структуры адресов эту особенность стоит учитывать: более специфичные маршруты размещайте выше, более общие – ниже. Например, если поставить path("<str:name>/", ...) выше path("new/", ...), то адрес /rooms/new/ не сработает: слово new совпадёт с переменной name, и запрос перехватит первый маршрут.

В корневом urls.py по умолчанию подключены адреса вида admin/ – это адреса «админ-зоны» проекта. Во избежание конфликтов не создавайте собственных адресов, начинающихся с этой строки.

При обработке однотипных адресов уникальную часть адреса сохраняют в переменную и передают её во view-функцию в качестве аргумента. Для этого в шаблоне адреса имя переменной заключают в треугольные скобки <>:

```
path("<int:room_id>/", views.room_detail, name="room_detail"),
```

Содержимое переменной можно проверить с помощью конвертеров пути. Наиболее востребованные:

Конвертер	Что ожидает	Пример
`str`	непустую строку без / (по умолчанию)	`<str:name>` → `about/company/`
`int`	целое неотрицательное число	`<int:pk>` → `catalog/1/`
`slug`	строку из латиницы, цифр, - и _	`<slug:category>` → `article/chto-takoe-slug/`

Если конвертер не указан явно, применяется `str`: шаблон `<name>/` идентичен шаблону `<str:name>/`.

В нашем проекте у объектов `Room` и `Booking` из ПР3 есть уникальные числовые идентификаторы, поэтому в маршрутах используется конвертер `<int:room_id>` и `<int:booking_id>`.

Для страниц помещений часто используют не числовые идентификаторы, а *слаги* – человекочитаемые части URL из латиницы, цифр, `-` и `_`. Адрес `/rooms/konferenc-zal/` понятнее, чем `/rooms/1/` – и для людей, и для поисковых роботов.

Чтобы перейти на *слаги*, нужно добавить в модель `Room` из ПР3 поле `slug` и заполнять его при создании помещения. Тогда маршрут примет вид:

```
path("<slug:room_slug>/", views.room_detail, name="room_detail"),
```

а view-функция будет принимать `room_slug` вместо `room_id`:

```
def room_detail(request, room_slug):
    ...
```

В этой работе мы пока используем `<int:room_id>`: в ПР3 у `Room` нет поля `slug`, и добавлять его в консольное приложение не требуется. Когда в следующих работах появится Django ORM и модель `Room`, поле `slug` можно будет добавить – и маршруты станут человекочитаемыми.

5.4.6. Страницы проекта

Страница формируется в `view`-функции как HTML-документ и возвращается в ответе. Чтобы все страницы проекта имели единый каркас, в файле `homepage/views.py` определите вспомогательную функцию `page()`: она собирает HTML-документ с кодировкой, заголовком вкладки, подключением Bootstrap и навигацией.

Функция `page()` размещается в приложении `homepage`, потому что именно главная страница задаёт общий вид всех остальных страниц – она точка входа в проект. Остальные приложения импортируют её.

```
from django.http import HttpResponse
```

```
def page(title, content):
    bootstrap = (
        "https://cdn.jsdelivr.net/npm/bootstrap@5.3.3"
        "/dist/css/bootstrap.min.css"
    )
    return f"""<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta    name="viewport"    content="width=device-width,    initial-
scale=1">
    <title>{title}</title>
    <link rel="stylesheet" href="{bootstrap}">
</head>
<body>
    <nav class="nav">
        <a class="nav-link" href="/">Главная</a>
        <a class="nav-link" href="/rooms/">Помещения</a>
        <a class="nav-link" href="/bookings/">Бронирования</a>
    </nav>
    <main class="container">{content}</main>
</body>
</html>"""
```

Функция `page()` подключает Bootstrap 5.3 через CDN и задаёт единый каркас всех страниц проекта. Копировать каркас в каждую view-функцию не требуется – достаточно вызвать `page()`, передав заголовок вкладки и содержимое страницы. Адрес стилевого файла Bootstrap вынесен в отдельную переменную, чтобы строка кода не превышала 79 символов.

Метатег `viewport` необходим для корректной работы адаптивности Bootstrap – без него сетка не будет подстраиваться под ширину экрана. Класс `container` ограничивает ширину содержимого и центрирует его: без него страница растянется на всю ширину большого монитора, и читать длинные строки будет неудобно.

Bootstrap работает через классы: достаточно добавить нужный класс к тегу – и элемент приобретёт стиль. Классы для типовых элементов (карточки, кнопки, списки) описаны в документации Bootstrap. Мы используем их и в каркасе (`page()`), и в содержимом страниц.

В файле `homepage/views.py` замените `view`-функцию `index()`:

```
def index(request):
    content = """
    <h1 class="display-4">Roomly</h1>
```

```
    <p class="lead">Система бронирования помещений.</p>
    <p>Основные разделы:</p>
    <a href="/rooms/" class="btn btn-primary me-2">Помещения</a>
    <a href="/bookings/" class="btn btn-secondary">Бронирования</a>
    """
    return HttpResponse(page("Roomly", content))
```

Классы `display-4` и `lead` делают заголовок и подзаголовок крупнее и заметнее, а `btn btn-primary` и `btn btn-secondary` превращают ссылки в кнопки.

Примечание. В следующей практической работе каркас страницы будет перенесён в отдельный шаблон (каталог `templates/` приложения), а данные – в контекст шаблона. Функция `page()` подготовила этот переход: останется заменить формирование строки на `render()`.

Такой подход – с формированием HTML прямо в Python-строке – временный. Он позволяет сосредоточиться на маршрутизации и передаче данных, не отвлекаясь на механизм шаблонов. В реальных Django-проектах HTML в view-функциях не пишут: разметка хранится в отдельных файлах-шаблонах, а `view` только готовит данные и выбирает шаблон. Когда в следующей работе мы перейдём на `render()` и `base.html`, функция `page()` исчезнет – её заменит шаблон, а каждая view-функция станет короче.

5.4.7. Обработка ошибки 404

В разделе 5.4.5 упоминалось: если адрес не обнаружился ни в одном `urlpatterns`, пользователь получает страницу с ошибкой `404 Page Not Found`. Пока `DEBUG = True`, Django показывает техническую страницу отладки: в ней перечислены все маршруты проекта и причина, по которой ни один из них не подошёл. Такая страница помогает разработчику искать ошибку, но не предназначена для посетителей сайта. Собственную страницу 404 можно проверить только при отключённой отладке.

Откройте файл `roomly/settings.py` и отключите режим отладки. Заодно заполните `ALLOWED_HOSTS` – при выключенной отладке Django принимает запросы только к адресам из этого списка:

```
DEBUG = False
ALLOWED_HOSTS = ["localhost", "127.0.0.1"]
```

Значения `localhost` и `127.0.0.1` означают локальную машину – этого достаточно для проверки на встроенном сервере разработки. Если список оставить пустым, Django при `DEBUG = False` ответит кодом 400 на любой запрос.

Стандартная страница Django при отключённой отладке – короткая служебная надпись «Not Found» без оформления проекта. Чтобы показать собственную страницу, подключите обработчик `handler404`. Обработчик объявляется строкой с путём к view-функции и размещается в корневом `urls.py` на уровне модуля, рядом с `urlpatterns`:

```
handler404 = "homepage.views.page_not_found"
```

Теперь добавьте в `homepage/views.py` view-функцию обработчика. Она собирает страницу через общий каркас `page()` и возвращает ответ с кодом 404:

```
def page_not_found(request, exception):
    content = """
    <h1 class="text-danger">404 – страница не найдена</h1>
    <p>Проверьте адрес или вернитесь на главную.</p>
    <a href="/" class="btn btn-primary">На главную</a>
    """
    return HttpResponse(
        page("404 – страница не найдена", content),
        status=404,
    )
```

Аргумент `exception` обязателен: при срабатывании обработчика Django передаёт в него объект исключения, приведшего к ошибке. На странице он не отображается – значение используется для журналирования и сложных сценариев обработки. Механизм тот же, что и в `room_detail()`: ответ формируется с кодом статуса 404. Разница в том, что `handler404` срабатывает автоматически для любого адреса, которому не нашлось маршрута, – отдельный код в каждой view-функции не требуется.

Страница обработчика вызывается не только при отсутствии маршрута. Если view-функция возбудит исключение `Http404` из модуля `django.http`, Django перехватит его и также покажет страницу 404. В этом проекте несуществующие объекты обрабатываются явным возвратом `HttpResponse` со статусом 404 (`room_detail()` в разделе 5.4.8) – оба способа корректны, выбор зависит от задачи.

Примечание. Настраивать обработку ошибки 404 в каждом приложении отдельно не требуется. Параметр `handler404` читается только из корневого `urls.py` проекта, поэтому обработчик объявляется один раз и действует для всех приложений сразу: Django перехватывает исключение `Http404`, возбуждённое в view-функции любого приложения, и показывает страницу обработчика. На уровне отдельного приложения остаётся лишь решить, когда запрошенные данные считаются не найденными, и сообщить об этом – возбудить `Http404` или вернуть ответ со статусом 404, как `room_detail()` приложения `rooms`. Объявление `handler404` внутри `urls.py` приложения не действует: Django игнорирует его и использует только обработчик из корневого модуля маршрутизации.

Сохраните изменения и откройте в браузере заведомо несуществующий адрес, например `http://127.0.0.1:8000/nonexistent/`. Вместо технической страницы отладки отобразится собственная страница проекта: заголовок, пояснение и кнопка возврата на главную.

Примечание. В режиме разработки значение `DEBUG = True` обычно сохраняют: технические страницы ошибок упрощают поиск проблем. Проверив собственную страницу 404, верните `DEBUG = True` – настройка `handler404` сохранится и заработает на рабочем сервере, где режим отладки отключён. Кроме того, при `DEBUG = False` встроенный сервер разработки не раздаёт статические файлы проекта.

5.4.8. Данные проекта ПР3 в веб-интерфейсе

Веб-страницы должны формироваться на основе данных приложения, а не представлять собой набор несвязанных статических HTML-файлов. Список помещений загружается из `data/rooms.json` средствами проекта ПР3.

В ПР3 реализована не только загрузка данных, но и логика работы с ними. Не вся эта логика используется в веб-интерфейсе на данном этапе: функции, изменяющие данные, требуют форм и POST-запросов, которые будут рассмотрены в следующих работах.

Функция ПР3	Назначение	Судьба в ПР5
`load_rooms()`	загрузить помещения из JSON	используется
`load_bookings()`	загрузить бронирования из JSON	используется
`load_users()`	загрузить пользователей из JSON	используется
`find_room_by_id()`	найти помещение по id	**new: используется**
`find_booking_by_id()`	найти бронирование по id	**new: используется**
`is_room_available()`	проверить доступность помещения	используется
`add_room()`	добавить помещение	отложена – нужна форма
`find_room()`	поиск помещений по названию	отложена – нужна форма поиска
`check_room_capacity()`	проверить вместимость	отложена – нужна форма
`filter_rooms_by_capacity()`	фильтр по вместимости	отложена – нужна форма
`sort_rooms()`	сортировка помещений	отложена – нужна форма
`create_booking()`	создать бронирование	отложена – нужна форма
`cancel_booking()`	отменить бронирование	отложена – нужна форма
`get_booking_status()`	текстовый статус	отложена – статус выводится через поле is_cancelled
`show_rooms()`	вывод в консоль	не используется – `print` не для веба
`show_bookings()`	вывод в консоль	не используется – `print` не для веба

В этой работе используются функции загрузки данных (`load_rooms()`, `load_users()`, `load_bookings()`), поиска объектов по идентификатору (`find_room_by_id()`, `find_booking_by_id()`) и проверки доступности помещения (`is_room_available()`).

Функции поиска по идентификатору используются при формировании страниц деталей. Идентификатор передаётся в URL, представление получает соответствующий объект `Room` или `Booking`, а затем формирует страницу на основе его атрибутов. Если объект с указанным идентификатором не найден, представление возвращает ответ с кодом HTTP 404.

Остальная логика ПР3 сохраняется в проекте и будет подключаться к веб-интерфейсу по мере появления соответствующих пользовательских действий и форм.

В файл rooms/views.py обновите импорты:

```
from datetime import date

from django.http import HttpResponse

from homepage.views import page
from models.bookings import is_room_available
from models.rooms import find_room_by_id
from storage import load_bookings, load_rooms, load_users
```

Замените view-функцию rooms(): она получает объекты Room и формирует страницу:

```
def rooms(request):
    items = ""
    for room in load_rooms("data/rooms.json"):
        text = f"{room.name} – вместимость {room.capacity} мест"
        items += f'<li class="list-group-item">{text}</li>'
    content = f"""
    <h1>Помещения</h1>
    <ul class="list-group">{items}</ul>
    """
    return HttpResponse(page("Roomly – помещения", content))
```

Классы list-group и list-group-item оформляют список помещений в виде аккуратных блоков с разделителями.

Для перехода к отдельному помещению используется маршрут приложения с числовым идентификатором (префикс rooms/ уже подключён в корневом urls.py):

```
path("<int:room_id>/", views.room_detail, name="room_detail")
```

Здесь <int:room_id> означает, что из URL передаётся целочисленный идентификатор помещения в параметр room_id.

Замените view-функцию room_detail(): страница помещения использует функцию find_room_by_id(), реализованную в ПР3. Если помещение не найдено, возвращается ответ с кодом 404:

```
def room_detail(request, room_id):
    rooms_list = load_rooms("data/rooms.json")

    room = find_room_by_id(rooms_list, room_id)

    if room is None:
        content = """
        <h1 class="text-danger">Помещение не найдено</h1>
        <a href="/rooms/" class="btn btn-outline-secondary">
```

```
        ← к списку помещений
    </a>
    """

    return HttpResponse(
        page("Помещение не найдено", content),
        status=404,
    )
users_list = load_users("data/users.json")
bookings_list = load_bookings(
    "data/bookings.json",
    rooms_list,
    users_list,
)
available = is_room_available(
    bookings_list,
    room,
    date.today(),
)
status = "доступно" if available else "занято"
badge = "bg-success" if available else "bg-danger"

content = f"""
<div class="card">
    <div class="card-body">
        <h5 class="card-title">{room.name}</h5>

        <p class="card-text">
            <strong>ID:</strong> {room.id}
        </p>

        <p class="card-text">
            <strong>Вместимость:</strong> {room.capacity} мест
        </p>
        <p class="card-text">
            Доступность на текущую дату:
            <span class="badge {badge}">
                {status}
            </span>
        </p>

        <a href="/rooms/"
           class="btn btn-outline-secondary">
            ← к списку помещений
        </a>
    </div>
```

```
    </div>
    """

    return HttpResponse(
        page(room.name, content),
        status=200,
    )
```

Функция `find_room_by_id()` выполняет поиск объекта `Room` по идентификатору:

```
def find_room_by_id(
    rooms: List[Room],
    room_id: int,
) -> Optional[Room]:
    """Найти помещение по идентификатору."""
    for room in rooms:
        if room.id == room_id:
            return room

    return None
```

Функция возвращает объект `Room`, если помещение найдено, и `None`, если помещения с указанным идентификатором нет. Поэтому представление может проверить результат поиска и вернуть HTTP 404.

После получения объекта `Room` представление загружает бронирования и передаёт их в функцию `is_room_available()`. Проверка выполняется для текущей даты, полученной с помощью `date.today()`.

Класс `card` оформляет информацию о помещении в виде карточки. Бейдж с классом `bg-success` или `bg-danger` показывает его текущий статус, а класс `text-danger` используется для сообщения об ошибке.

Список бронирований формируется аналогично по данным `data/bookings.json` (функция `load_bookings()`). В файл `bookings/views.py` обновите импорты и замените view-функции:

```
from django.http import HttpResponse

from homepage.views import page
from models.bookings import find_booking_by_id
from storage import load_bookings, load_rooms, load_users

def bookings(request):
    rooms_list = load_rooms("data/rooms.json")
    users_list = load_users("data/users.json")
    bookings_list = load_bookings(
```

```
        "data/bookings.json",
        rooms_list,
        users_list,
    )

    items = ""
    for booking in bookings_list:
        status = "отменено" if booking.is_cancelled else "активно"
        badge = "bg-secondary" if booking.is_cancelled else "bg-success"
        items += f"""
        <li class="list-group-item d-flex justify-content-between">
            <a href="/bookings/{booking.id}/">
                {booking.room.name} - {booking.booking_date}
            </a>
            <span class="badge {badge}">
                {status}
            </span>
        </li>
        """
    content = f"""
    <h1>Бронирования</h1>
    <ul class="list-group">
        {items}
    </ul>
    """
    return HttpResponse(
        page("Roomly - бронирования", content)
    )
```

Для перехода к отдельному бронированию используется маршрут приложения (префикс `bookings/` уже подключён в корневом urls.py):

```
path("<int:booking_id>/", views.booking_detail, name="booking_detail")
```

Параметр `<int:booking_id>` передаёт числовой идентификатор бронирования в view-функцию.

Страница отдельного бронирования:

```
def booking_detail(request, booking_id):
    rooms_list = load_rooms("data/rooms.json")

    users_list = load_users("data/users.json")
    bookings_list = load_bookings(
        "data/bookings.json",
        rooms_list,
        users_list,
    )
```

```
    booking = find_booking_by_id(
        bookings_list,
        booking_id,
    )
    if booking is None:
        content = """
        <h1 class="text-danger">
            Бронирование не найдено
        </h1>
        <a href="/bookings/"
           class="btn btn-outline-secondary">
            ← к списку бронирований
        </a>
        """
        return HttpResponse(
            page("Бронирование не найдено", content),
            status=404,
        )

    status = "отменено" if booking.is_cancelled else "активно"
    badge = "bg-secondary" if booking.is_cancelled else "bg-success"
    content = f"""
    <div class="card">
        <div class="card-body">
            <h5 class="card-title">
                Бронирование №{booking.id}
            </h5>
            <p class="card-text">
                Помещение: {booking.room.name}
            </p>
            <p class="card-text">
                Дата: {booking.booking_date}
            </p>
            <p class="card-text">
                Пользователь: {booking.user.name}
            </p>
            <p class="card-text">
                Статус:
                <span class="badge {badge}">
                    {status}
                </span>
            </p>
            <a href="/bookings/"
               class="btn btn-outline-secondary">
                ← к списку бронирований
            </a>
```

```
        </div>
    </div>
    """
    return HttpResponse(
        page(
            f"Бронирование №{booking.id}",
            content,
        ),
        status=200,
    )
```

Функция `find_booking_by_id()` реализована по аналогии с `find_room_by_id()`. Добавьте её в модуль `models/bookings.py`:

```
def find_booking_by_id(
    bookings: List[Booking],
    booking_id: int,
) -> Optional[Booking]:
    """Найти бронирование по идентификатору."""
    for booking in bookings:
        if booking.id == booking_id:
            return booking
    return None
```

Функция возвращает объект `Booking`, если бронирование найдено, и `None`, если бронирования с указанным идентификатором нет. В последнем случае представление возвращает страницу с сообщением об ошибке и HTTP-кодом 404.

В списке и карточке бронирования используется поле `is_cancelled`. На его основе формируется текстовый статус и выбирается соответствующий цветной бейдж.

В карточке бронирования выводятся связанные объекты:

```
booking.room.name
booking.user.name
```

Таким образом, объект `Booking` продолжает использовать связи с объектами `Room` и `User`, сформированные в предметной модели ПР3.

На данном этапе Django-представления не дублируют предметную логику проекта. Они выполняют следующие задачи:

- получают HTTP-запрос;
- загружают данные средствами ПР3;
- при необходимости находят конкретный объект по ID;
- вызывают функции предметной модели для необходимых проверок;
- формируют HTML-ответ.

Таким образом, веб-слой использует уже существующие классы и функции ПР3, а не заменяет их отдельной реализацией.

Примечание. Функции ПР3 возвращают объекты `Room` и `Booking`, поэтому формирование веб-страниц выполняется по атрибутам объектов, а не по словарям. Связи между объектами предметной области сохраняются: для бронирования доступны связанные объекты помещения и пользователя (`booking.room`, `booking.user`). На следующих этапах источником данных станет база данных, доступ к которой будет организован средствами Django ORM. Текущий веб-интерфейс используется как основа для дальнейшей миграции от хранения данных в JSON-файлах к хранению данных в базе данных.

5.5. Обновление README.md

После завершения ПР5 необходимо обновить README.md. В README следует отразить:

- назначение проекта и предметную область;
- используемый фреймворк Django;
- структуру Django-проекта;
- реализованные страницы и их URL;
- назначение основных view-функций;
- используемые HTML/CSS/Bootstrap;
- инструкцию по запуску;
- текущее состояние проекта.

Пример файла README.md проекта `Roomly` после выполнения ПР5:

```
# Roomly – система бронирования помещений

## Назначение

Roomly – приложение для управления помещениями и их бронированием.
Консольная версия приложения (ПР3) дополнена веб-интерфейсом на Django (ПР5).

Приложение позволяет:
* просматривать помещения в браузере;
* просматривать информацию о конкретном помещении;
* просматривать список бронирований в браузере;
* просматривать информацию о конкретном бронировании;
* проверять доступность помещения на определённую дату;
* находить помещения по названию;
* проверять вместимость помещений;
* создавать бронирования;
* отменять бронирования;
* добавлять новые помещения;
* сортировать помещения;
* хранить данные между запусками приложения.

## Используемые технологии

* Python 3;
* Django 5.2 – веб-фреймворк;
* HTML – разметка веб-страниц;
* CSS и Bootstrap 5.3 – оформление страниц;
* pytest – автоматизированные тесты;
* flake8 – проверка качества кода.

## Целевая аудитория

Пользователи, которым необходимо бронировать помещения для проведения занятий, встреч и мероприятий.

## Предметная область

Основными объектами предметной области являются:
* помещение;
* пользователь;
```

```
* бронирование.

Пользователь может создавать бронирования помещений на определённые даты.
Одно помещение не может иметь два активных бронирования на одну и ту же дату.
Отменённое бронирование не блокирует помещение и позволяет создать новое бронирование на эту дату.

## Основные сущности

### Room – помещение

Объект `Room` представляет помещение, доступное для бронирования.

Основные атрибуты:
* `id` – уникальный идентификатор помещения;
* `name` – название помещения;
* `capacity` – вместимость помещения.

Основные методы:
* `is_suitable_for()` – проверка соответствия помещения заданному количеству людей;
* `__str__()` – строковое представление помещения.

### User – пользователь

Объект `User` представляет пользователя системы.

Основные атрибуты:
* `id` – уникальный идентификатор пользователя;
* `name` – имя пользователя;
* `email` – адрес электронной почты.

Основные методы:
* `__str__()` – строковое представление пользователя;
* `from_data()` – создание объекта `User` из набора данных.

### Booking – бронирование

Объект `Booking` представляет бронирование помещения пользователем.

Основные атрибуты:
* `id` – уникальный идентификатор бронирования;
* `room` – объект `Room`;
* `booking_date` – дата бронирования;
* `user` – объект `User`;
* `is_cancelled` – признак отмены бронирования.

Основные методы:
* `cancel()` – отмена бронирования;
* `__str__()` – строковое представление бронирования.

## Страницы веб-интерфейса

* `/` – главная страница: описание приложения и навигация (view-функция `index()`);
* `/rooms/` – список помещений: объекты `Room` из `data/rooms.json` (view-функция `rooms()`);
* `/rooms/<int:room_id>/` – страница помещения (view-функция `room_detail()`);
* `/bookings/` – список бронирований: объекты `Booking` из `data/bookings.json` (view-функция `bookings()`);
* `/bookings/<int:booking_id>/` – страница бронирования (view-функция `booking_detail()`).

Если помещение или бронирование с указанным идентификатором не найдено, соответствующая страница возвращает код статуса 404.

## Основные операции

### Используется в веб-интерфейсе (ПР5)

* просмотр списка помещений;
* просмотр информации о помещении;
* просмотр списка бронирований;
* просмотр информации о бронировании;
* проверка доступности помещения на текущую дату.

### Отложено до следующих работ

Эти операции реализованы в ПР3 и работают в консольной версии. В веб-интерфейсе они появятся, когда будут добавлены формы и POST-запросы:

* добавление помещения;
* поиск помещения по названию;
* проверка вместимости;
* фильтрация помещений по вместимости;
* сортировка помещений;
* создание бронирования;
* отмена бронирования;
```

```
* создание пользователя;
* поиск пользователя.

## Хранение данных

Данные приложения хранятся в JSON-файлах:
* `data/rooms.json`;
* `data/users.json`;
* `data/bookings.json`.

При загрузке данные из JSON преобразуются в объекты `Room`, `User` и `Booking`.
При сохранении объекты преобразуются в данные JSON.

Для связи бронирования с помещением и пользователем в `bookings.json` используются их идентификаторы `room_id` и `user_id`.

## Структура проекта

```text
room_booking/
│
├── README.md
├── requirements.txt
├── manage.py
├── main.py
├── storage.py
├── utils.py
│
├── roomly/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
├── homepage/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── migrations/
│   ├── models.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
│
├── rooms/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── migrations/
│   ├── models.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
│
├── bookings/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── migrations/
│   ├── models.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
│
├── models/
│   ├── __init__.py
│   ├── rooms.py
│   ├── bookings.py
│   └── users.py
│
├── data/
│   ├── rooms.json
│   ├── bookings.json
│   └── users.json
│
└── tests/
    ├── test_rooms.py
    ├── test_bookings.py
    └── test_users.py
```
```

````
## Назначение модулей

### `main.py`

Консольная версия приложения (сохранена с ПР3).
Отвечает за:
* запуск консольного приложения;
* отображение меню;
* получение данных от пользователя;
* организацию пользовательских сценариев.

### `roomly/`

Пакет настроек Django-проекта:
* `settings.py` – настройки проекта;
* `urls.py` – маршрутизация верхнего уровня;
* `asgi.py`, `wsgi.py` – точки взаимодействия с сервером.

### `homepage/`

Django-приложение, отвечающее за главную страницу:
* `views.py` – функция `page()` (общий HTML-каркас) и view-функция `index()`;
* `urls.py` – маршрут главной страницы.

Функция `page()` размещена здесь, потому что главная страница задаёт общий вид всех остальных страниц – она точка входа в проект. Остальные приложения импортируют её.

### `rooms/`

Django-приложение, отвечающее за помещения:
* `views.py` – view-функции `rooms()` и `room_detail()`;
* `urls.py` – маршруты приложения;
* `models.py` – Django-модели (заполняется в следующих работах).

### `bookings/`

Django-приложение, отвечающее за бронирования:
* `views.py` – view-функции `bookings()` и `booking_detail()`;
* `urls.py` – маршруты приложения;
* `models.py` – Django-модели (заполняется в следующих работах).

### `models/`

Классы предметной области и функции работы с ними (пакет ПР3):
* `rooms.py` – класс `Room` и функции работы с помещениями;
* `users.py` – класс `User` и функции работы с пользователями;
* `bookings.py` – класс `Booking` и функции работы с бронированиями.

### `storage.py`

Отвечает за загрузку и сохранение данных.
Выполняет преобразование:
* JSON → объекты Python;
* объекты Python → JSON.

### `utils.py`

Содержит вспомогательные функции консольного ввода:
* `input_int()` – безопасный ввод целого числа;
* `input_date()` – безопасный ввод даты.

### `tests/`

Содержит автоматизированные тесты проекта.

## Запуск приложения

Веб-версия (Django):
```text
python manage.py runserver
```

После запуска открыть в браузере адрес:
```text
http://127.0.0.1:8000/
```

Консольная версия (сохранена):
```text
python main.py
```
````

```
## Тестирование

Для запуска автоматизированных тестов выполнить:
```text
pytest -v
```

Проверка качества кода

Для проверки соответствия кода требованиям PEP 8 используется `flake8`.
Команда проверки:
```text
flake8 .
```

Зависимости

Состав зависимостей в `requirements.txt`:
* `Django==5.2.*` – веб-фреймворк;
* `pytest` – автоматизированные тесты;
* `flake8` – проверка качества кода.

Git

Исходный код проекта хранится в Git-репозитории.
Для проверки состояния репозитория:
```text
git status
```

Текущее состояние и план развития

Реализовано (ПР5):
* Django-проект и три Django-приложения (`homepage`, `rooms`, `bookings`);
* маршрутизация URL двух уровней;
* view-функции и страницы;
* отображение данных предметной области (помещения и бронирования);
* проверка доступности помещения через логику ПР3;
* навигация между страницами;
* оформление Bootstrap 5.3.

Планируется:
* перенос сущностей `Room`, `User` и `Booking` в Django ORM;
* подключение базы данных;
* переход на шаблоны (`templates/`) и отказ от функции `page()`;
* формы создания и редактирования бронирований;
* использование отложенной логики ПР3 (`create_booking()`, `cancel_booking()`, `add_room()`, поиск и сортировка);
* разработка API;
* контейнеризация приложения;
* настройка CI/CD.
```

Логика работы с сущностями проекта распределена между файлами. В таблице приведены функции каждой сущности, файлы их расположения и назначение. Состав функций сущностей соответствует результату ПР3; функции веб-слоя добавлены в ПР5.

Сущность Room – помещение (ПР3):

Имя функции	Файл расположения	Назначение
`Room` (класс)	`models/rooms.py`	помещение: атрибуты id, name, capacity
`is_suitable_for()`	`models/rooms.py`	проверка соответствия помещения заданному количеству людей
`info` (`@property`)	`models/rooms.py`	краткое описание помещения для отображения
`__str__()`	`models/rooms.py`	строковое представление помещения
`add_room()`	`models/rooms.py`	создание объекта Room и добавление его в коллекцию
`find_room()`	`models/rooms.py`	поиск объектов Room по названию
`find_room_by_id()`	`models/rooms.py`	поиск объекта Room по идентификатору; используется веб-страницей помещения

check_room_capacity()	models/rooms.py	поиск помещения и проверка его вместимости
filter_rooms_by_capacity()	models/rooms.py	отбор объектов Room по вместимости
sort_rooms()	models/rooms.py	сортировка коллекции объектов Room
show_rooms()	models/rooms.py	вывод информации об объектах Room

Сущность User – пользователь (ПР3)

Имя функции	Файл расположения	Назначение
User (класс)	models/users.py	пользователь: атрибуты id, name, email
from_data()	models/users.py	создание объекта User из набора данных JSON
__str__()	models/users.py	строковое представление пользователя
add_user()	models/users.py	создание объекта User и добавление его в коллекцию
find_user()	models/users.py	поиск объектов User по имени или адресу электронной почты
find_user_by_id()	models/users.py	поиск объекта User по идентификатору
show_users()	models/users.py	вывод информации об объектах User

Сущность Booking – бронирование (ПР3)

Имя функции	Файл расположения	Назначение
Booking (класс)	models/bookings.py	бронирование: атрибуты id, room, user, booking_date, is_cancelled
cancel()	models/bookings.py	отмена бронирования (установка признака is_cancelled)
__str__()	models/bookings.py	строковое представление бронирования
find_booking_by_id()	models/bookings.py	поиск объекта Booking по идентификатору (добавлена в ПР5); используется веб-страницей бронирования
is_room_available()	models/bookings.py	проверка доступности помещения с учётом активных бронирований; используется веб-страницей помещения
create_booking()	models/bookings.py	проверка доступности, создание объекта Booking и добавление его в коллекцию
cancel_booking()	models/bookings.py	поиск бронирования и вызов его метода cancel()
get_booking_status()	models/bookings.py	формирование сообщения о доступности помещения
show_bookings()	models/bookings.py	вывод информации об объектах Booking

Хранение данных (storage.py, ПР3)

Имя функции	Файл расположения	Назначение
load_rooms()	storage.py	загрузка data/rooms.json: JSON → объекты Room; используется веб-страницами помещений
save_rooms()	storage.py	сохранение data/rooms.json: объекты Room → JSON
load_users()	storage.py	загрузка data/users.json: JSON → объекты User
save_users()	storage.py	сохранение data/users.json: объекты User →

		JSON
load_bookings()	storage.py	загрузка data/bookings.json: JSON → объекты Booking; используется веб-страницами помещений и бронирований
save_bookings()	storage.py	сохранение data/bookings.json: объекты Booking → JSON

Веб-слой (ПР5)

Приложение homepage

Имя функции	Файл расположения	Назначение
page()	homepage/views.py	единый каркас HTML-страницы: кодировка, заголовок, Bootstrap 5.3, навигация
index()	homepage/views.py	главная страница /

Приложение rooms

Имя функции	Файл расположения	Назначение
rooms()	rooms/views.py	страница /rooms/: список помещений из data/rooms.json
room_detail()	rooms/views.py	страница /rooms/<int:room_id>/; если помещение не найдено – код статуса 404

Приложение bookings

Имя функции	Файл расположения	Назначение
bookings()	bookings/views.py	страница /bookings/: список бронирований из data/bookings.json
booking_detail()	bookings/views.py	страница /bookings/<int:booking_id>/; если бронирование не найдено – код статуса 404

Примечание. Функции сущностей и хранилища используются одновременно консольной и веб-версией приложения: view-функции ПР5 вызывают load_rooms(), load_users(), load_bookings(), find_room_by_id() и find_booking_by_id() вместо консольных сценариев main.py. Предметная область ПР3, таким образом, сохраняется в веб-интерфейсе.

5.6. Проверка и фиксация результата

5.6.1. Проверка Django-приложения

Перед первым запуском приложения применить встроенные миграции Django:

```
python manage.py migrate
```

Команда создаёт необходимые служебные таблицы базы данных и устраняет предупреждение о наличии неприменённых миграций.

После применения миграций выполнить системную проверку проекта:

```
python manage.py check
```

После этого запустить development-сервер:

```
python manage.py runserver
```

При отсутствии ошибок и предупреждений открыть в браузере адрес:

```
http://127.0.0.1:8000/
```

Проверить:

- запуск Django без ошибок;

- открытие главной страницы;
- работу каждого настроенного URL;
- переходы между страницами по ссылкам навигации;
- отображение данных предметной области;
- корректную обработку динамических URL;
- корректную обработку несуществующих идентификаторов (код 404);
- отсутствие ошибок в консоли сервера.

Для динамических маршрутов необходимо проверить как минимум по одному существующему идентификатору:

- `/rooms/1/` – страница помещения;
- `/bookings/1/` – страница бронирования.

Также проверьте несуществующие идентификаторы:

- `/rooms/999/` – должна вернуться страница с сообщением «Помещение не найдено» и кодом 404;
- `/bookings/999/` – должна вернуться страница с сообщением «Бронирование не найдено» и кодом 404.

5.6.2. Проверка маршрутов и страниц

Для каждой страницы проверяется соответствие URL → view → response. Полный маршрут запроса конкретной страницы показан на рис. 7.

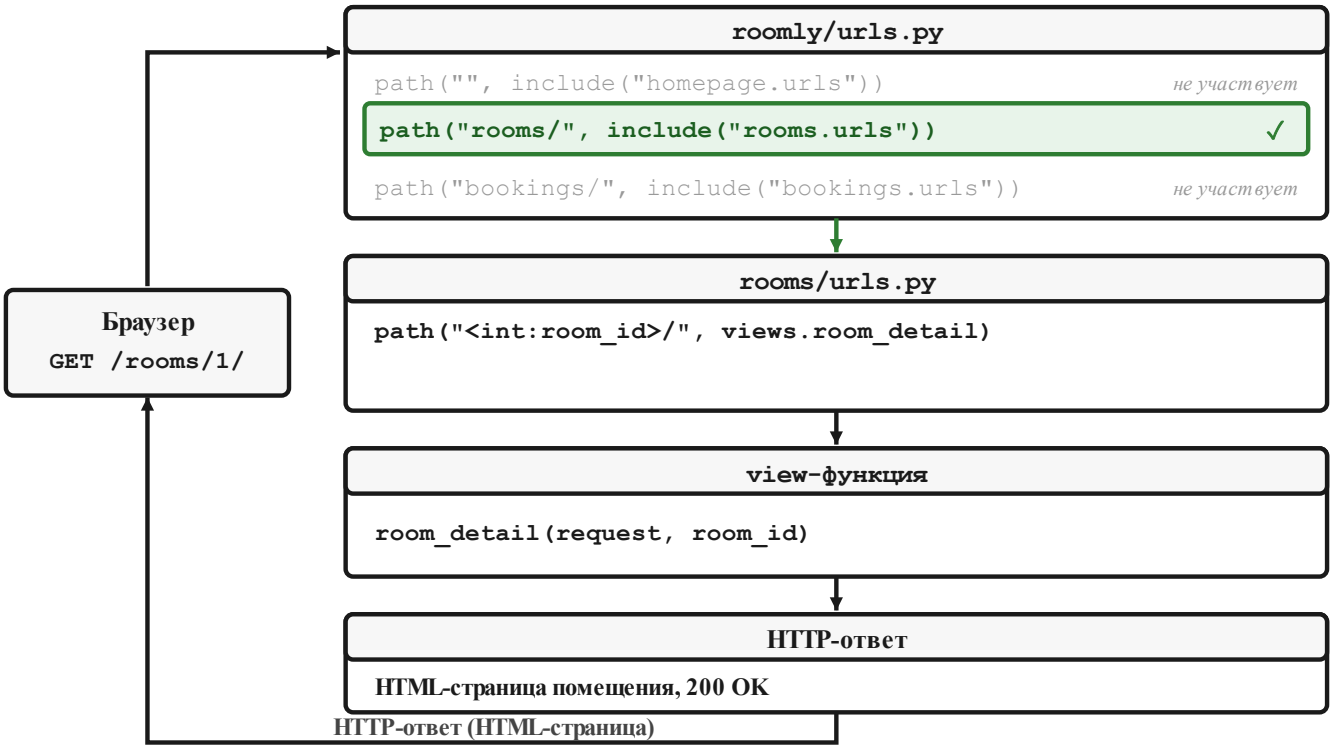


Рисунок 7 – Маршрут запроса `/rooms/1/`

Проверить HTML-страницы:

- корректную структуру HTML-документа;
- заголовок вкладки (`title`) и заголовки разделов;
- корректное отображение текста;
- работоспособность ссылок навигации;
- отсутствие очевидных ошибок в разметке;
- единообразие оформления страниц.

Обратите внимание на три приложения – навигация должна работать между всеми:

Страница	URL	Что проверить
Главная	`/`	описание приложения, кнопки-ссылки на разделы
Помещения	`/rooms/`	список помещений из `data/rooms.json`
Помещение	`/rooms/1/`	карточка помещения, статус доступности
Бронирования	`/bookings/`	список бронирований, статусы-бейджи
Бронирование	`/bookings/1/`	карточка бронирования, связанные объекты

Проверьте и обработку несуществующего адреса: установите `DEBUG = False` (раздел 5.4.7), откройте `/nonexistent/` – должна отобразиться собственная страница `404`. После проверки верните `DEBUG = True`.

Проверьте оформление: подключение Bootstrap, применение стилей, корректное отображение страниц в браузере.

При проверке адаптивности уменьшите ширину окна браузера – страницы должны корректно перестраиваться благодаря метатегу `viewport` и классу `container`.

5.6.3. Проверка качества кода

Выполнить проверку Python-кода и тестов – те же инструменты, что в предыдущих работах:

```
flake8 .
pytest -v
```

Примечание. flake8 проверяет все Python-файлы проекта, включая созданные Django автоматически (`settings.py`, `admin.py`, `models.py`, `tests.py` приложений) и каталог виртуального окружения `venv/`. Каталог `venv/` исключается из проверки: `flake8 . --extend-exclude=venv`. Неиспользуемые импорты в сгенерированных файлах можно удалить.

При наличии замечаний необходимо устранить их и повторить проверку. Автоматизированные тесты ПР3 должны проходить без изменений: консольная часть проекта сохранена. Особое внимание уделяется:

- структуре импортов;
- отступам и длине строк;
- именованию view-функций;
- отсутствию дублирования кода;
- структуре Django-приложения.

5.6.4. Проверка изменений Git

Проверить состояние репозитория и зафиксировать результат:

```
git status
git diff
git add .
git commit -m "PR5: add Django project and web pages"
```

Каждый коммит, относящийся к данной практической работе, должен начинаться с метки `PR5:`. При использовании удаленного репозитория отправить изменения:

```
git push
```

6. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

К завершению ПР5 должны быть получены следующие результаты. Используйте этот перечень для самопроверки перед переходом к контролю: каждый пункт должен быть подтвержден демонстрацией или содержимым репозитория.

Материалы Яндекс Практикума. *Раздел 04. «Основы веб-разработки»*:

Тема «Создание проекта»:

- Введение в спринт;
- Фреймворк Django;
- Создание Django-проекта;
- Приложения в Django-проекте.

Тема «Пути и view-функции»:

- Протокол HTTP: стандарт для запросов и ответов;
- Планирование адресов и конвертеры путей;
- Распределение адресов по приложениям;
- View-функции: от запроса до ответа.

Тема «Вёрстка для бэкендера»:

- Веб-страницы: язык HTML;
- Структура HTML-документа.
- CSS: стили, селекторы и синтаксис;
- Иерархия HTML-документа и наследование стилей;
- Фреймворк Bootstrap: быстрая HTML-верстка.

Индивидуальный проект:

- проект ПР3 сохранен и продолжен;
- создан Django-проект;
- созданы Django-приложения и они подключены к проекту;
- настроена маршрутизация;

- реализованы view-функции;
- создана главная страница;
- созданы страницы основных объектов предметной области;
- настроены динамические URL;
- организована навигация между страницами;
- используется HTML;
- используется CSS и/или Bootstrap;
- веб-интерфейс соответствует предметной области проекта;
- README.md обновлен;
- приложение запускается без ошибок;
- изменения зафиксированы в Git.

На данном этапе не требуется

- переносить сущности в Django ORM;
- создавать базу данных проекта;
- реализовывать Django Forms;
- реализовывать регистрацию и авторизацию пользователей;
- реализовывать создание и редактирование объектов через веб-формы.

Эти возможности изучаются на последующих практических работах.

7. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ

Контроль выполнения ПР5 включает:

- проверку материалов Яндекс Практикума;
- проверку структуры Django-проекта;
- запуск приложения;
- демонстрацию основных URL;
- демонстрацию view-функций;
- проверку HTML/CSS/Bootstrap;
- демонстрацию индивидуального проекта;
- проверку README.md;
- проверку Git;
- ответы на контрольные вопросы.

7.1. Чек-лист выполнения материалов Яндекс Практикума

Студент предъявляет преподавателю результат выполнения следующих материалов. Отмечайте пункты по мере выполнения – чек-лист используется и при самопроверке, и при контроле со стороны преподавателя. Перечень тем и уроков соответствует разделу 4.

№	Тема	Урок (материал)	Выполнено
1	Создание проекта	«Введение в спринт»	☐
2	Создание проекта	«Фреймворк Django»	☐
3	Создание проекта	«Создание Django-проекта»	☐
4	Создание проекта	«Приложения в Django-проекте»	☐
5	Пути и view-функции	«Протокол HTTP: стандарт для запросов и ответов»	☐
6	Пути и view-функции	«Планирование адресов и конвертеры путей»	☐
7	Пути и view-функции	«Распределение адресов по приложениям»	☐
8	Пути и view-функции	«View-функции: от запроса до ответа»	☐
9	Вёрстка для бэкендера	«Веб-страницы: язык HTML»	☐
10	Вёрстка для бэкендера	«Структура HTML-документа»	☐
11	Вёрстка для бэкендера	«CSS: стили, селекторы и синтаксис»	☐
12	Вёрстка для бэкендера	«Иерархия HTML-документа и наследование стилей»	☐
13	Вёрстка для бэкендера	«Фреймворк Bootstrap: быстрая HTML-верстка»	☐

7.2. Чек-лист выполнения индивидуального проекта

Чек-лист доработки индивидуального проекта:

№	Требование	Выполнено
1	Проект ПР3 сохранен и продолжен	☐
2	Создан Django-проект	☐
3	Созданы Django-приложения	☐
4	Приложения подключены к проекту	☐
5	Настроена маршрутизация	☐
6	Настроена обработка ошибки 404	☐
7	Реализованы view-функции	☐
8	Создана главная страница	☐
9	Созданы страницы предметной области	☐
10	Используются динамические URL	☐
11	Организована навигация	☐
12	Используется HTML	☐
13	Используется CSS и/или Bootstrap	☐
14	Сохраняется предметная область проекта	☐
15	Приложение запускается без ошибок	☐
16	Реализованные URL проверены	☐
17	README.md обновлен	☐
18	Код проверен (flake8, pytest)	☐
19	Результат зафиксирован в Git	☐

7.3. Критерии оценки

№	Критерий	Содержание
1	Материалы Практикума	Выполнены предусмотренные материалы раздела 04
2	Django-проект	Проект создан и запускается
3	Django-приложение	Приложения созданы и подключены
4	URL-маршрутизация	URL связаны с соответствующими view; настроена обработка ошибки 404
5	View-функции	Представления корректно обрабатывают запросы

№	Критерий	Содержание
6	HTTP	Продемонстрировано понимание цикла запрос–ответ
7	HTML	Созданы корректные веб-страницы
8	CSS/Bootstrap	Реализовано оформление интерфейса
9	Индивидуальный проект	Веб-интерфейс соответствует предметной области
10	Навигация	Основные страницы связаны между собой
11	README	Документация соответствует текущему состоянию проекта
12	Git	Результат работы зафиксирован в репозитории

7.4. Требования к отчету

Объем отчета – 1–2 страницы, не считая титульной информации. Отчет должен содержать:

1. **Общие сведения:**

- название практической работы;
- ФИО;
- группа;
- название индивидуального проекта.

2. **Выполненные материалы Яндекс Практикума:**

Раздел	Выполненная тема
Раздел 04. «Основы веб-разработки»	«Создание проекта»
	«Пути и view-функции»
	«Вёрстка для бэкендера»

3. **Доработка индивидуального проекта.** Указать:

- название проекта;
- созданный Django-проект;
- созданные Django-приложения;
- реализованные URL и страницы;
- краткое описание выполненной работы;
- ссылку на Git-репозиторий.

4. **Результат работы.** Кратко указать, что в результате выполнения ПР5:

- созданы Django-приложения;
- настроена маршрутизация;
- реализованы view-функции;
- создан веб-интерфейс;
- реализованы основные страницы проекта;
- настроена навигация;
- использованы HTML, CSS и/или Bootstrap;
- сохранена предметная область индивидуального проекта.

5. **Вывод.**

Сделать краткий вывод о результатах выполнения практической работы.

8. КОНТРОЛЬНЫЕ ВОПРОСЫ

При подготовке к защите необходимо уметь ответить на контрольные вопросы и продемонстрировать соответствующий участок проекта.

Django и веб-разработка

1. Что такое веб-приложение?
2. Что такое фреймворк?
3. Что такое Django?
4. Чем фреймворк отличается от библиотеки?
5. Что такое Django-проект?
6. Что такое Django-приложение?
7. Чем Django-проект отличается от Django-приложения?
8. Для чего используется `manage.py`?
9. Для чего используется `settings.py`?
10. Для чего используется `urls.py`?
11. Для чего используется `views.py`?
12. Что такое development-сервер Django?

HTTP

13. Что такое HTTP?
14. Что такое HTTP-запрос?
15. Что такое HTTP-ответ?
16. Из каких основных частей состоит HTTP-запрос?
17. Из каких основных частей состоит HTTP-ответ?
18. Что такое URL?
19. Что такое маршрут?
20. Что происходит после обращения пользователя к URL Django-приложения?
21. Что такое HTTP-метод?
22. Чем GET отличается от POST?

URL и view

23. Для чего используется `path()`?
24. Что такое конвертер пути?
25. Для чего используется `<int:id>`?
26. Что такое view-функция?
27. Какой параметр получает view-функция Django?

28. Что должна вернуть view-функция?

29. Что такое HttpResponse?

30. Как передать параметр из URL в view?

31. Зачем использовать имена URL-маршрутов?

32. Почему URL-маршрутизацию следует отделять от логики обработки данных?

HTML, CSS и Bootstrap

33. Что такое HTML?

34. Для чего используется HTML-документ?

35. Из каких основных частей состоит HTML-документ?

36. Что такое HTML-элемент?

37. Что такое CSS?

38. Для чего используются CSS-селекторы?

39. Что такое наследование стилей?

40. Что такое Bootstrap?

41. Для чего используется Bootstrap?

42. Какие преимущества дает использование CSS-фреймворка?

Индивидуальный проект

43. Какая предметная область реализована в индивидуальном проекте?

44. Какие страницы реализованы?

45. Какие URL соответствуют этим страницам?

46. Какие view-функции реализованы?

47. Как запрос пользователя проходит от URL до HTML-страницы?

48. Какие элементы проекта были перенесены из консольного приложения ПР3?

49. Почему на ПР5 еще не выполняется полный переход на Django ORM?

50. Как будет изменяться проект после подключения базы данных?

51. Какие данные пока используются для отображения страниц?

52. Какие возможности проекта будут реализованы на следующих практических работах?

9. ИТОГ

В ПР5 консольный индивидуальный проект переводится на начальный этап веб-разработки с использованием Django. В результате:

- изучены назначение и структура Django;
- создан Django-проект;
- созданы Django-приложения;

- изучен принцип обработки HTTP-запросов;
- настроена маршрутизация URL;
- реализованы view-функции;
- созданы веб-страницы;
- изучены HTML и CSS;
- использован Bootstrap;
- организована навигация между страницами;
- предметная область индивидуального проекта отображена в веб-интерфейсе;
- сохранена связь с результатами ПР3: классы, JSON-хранилище, тесты и консольная версия продолжают работать;
- обновлена документация проекта;
- результат зафиксирован в Git.

ПР5 формирует основу для дальнейшего перехода от простого веб-интерфейса к полноценному Django-приложению.

Следующий этап – работа с Django-шаблонами, контекстом и наследованием шаблонов, статическими файлами, а также дальнейшая доработка индивидуального проекта: перенос сущностей `Room`, `User` и `Booking` в Django ORM.